

Батраева И.А.

Объектно-ориентированное программирование
Часть 1. Основы разработки на языке C++

Учебное пособие

Саратов 2019

УДК 004.42.045

ББК 32.973

Б 28

Батраева И.А.

Объектно-ориентированное программирование: Учебное пособие. Ч. 1.
Основы разработки на языке С++. – Саратов: Издательство ЗАО ЛА
«Научная книга», 2019. – 40 с.

Данное учебное пособие посвящено изучению основ объектно-ориентированного программирования. Пособие состоит из двух частей. В первой части рассматривается проектирование программного обеспечения на языке С++, основные принципы ООП, принципы преобразования типов обработки исключительных ситуаций.

Пособие предназначено для студентов всех специальностей и направлений подготовки, связанных с информационными технологиями.

Рецензент

доктор физико-математических наук, доцент Д.В. Кондратов

Работа издана в авторской редакции

СОДЕРЖАНИЕ

1 История развития и основные понятия объектно-ориентированного программирования	4
2 Основные принципы ООП.....	7
2.1 Абстракция и инкапсуляция	7
2.2 Иерархия, наследование и полиморфизм	12
2.3 Конструкторы, деструкторы, дружественные функции, статические члены класса.....	26
2.4 Преобразование типов	32
Заключение	34
Список использованных источников	35
Приложение.....	36
Приложение А. Построение абстракций классов.....	36
Приложение В. Задания по проектированию иерархии классов.....	37
Приложение С. Обработка исключительных ситуаций.....	38

1 История развития и основные понятия объектно-ориентированного программирования

Современные программные системы, эксплуатирующиеся в производстве, гораздо больше и сложнее тех, что были еще несколько лет назад. Эта тенденция продолжается на самом деле с начала 60-х годов, когда стало возможным говорить о промышленном программировании. Первые языки программирования были созданы для ухода от программирования в машинных кодах и на ассемблере и обладали существенным недостатком – разные данные не изолировались друг от друга и от различных подпрограмм. Поэтому, как только система начинала расти, лавинообразно накапливалось количество ошибок, конфликтов между данными и подпрограммами. Причем подпрограммы, по сути, были просто выделенными участками кода, которые также могли конфликтовать друг с другом. К числу таких языков относятся FORTRAN I, ALGOL-58, Flowmatic, IPL V.

С середины 60-х годов понимание этих проблем привело к появлению новых версий или новых языков, в которых большое внимание стало уделяться т.н. алгоритмическим абстракциям, и первой такой абстракцией стала процедурная абстракция. Теперь предполагалось, что задачу можно разделять на отдельные фрагменты по смыслу решаемых подзадач, использовать ранее созданные процедуры в качестве основы для проектирования новых программ, но на этом этапе продолжал отсутствовать контроль над взаимодействием данных и процедур. Примерами таких языков являются ALGOL-60, COBOL, Lisp.

К концу 60-х и началу 70-х годов в языки программирования проникла идея модульной структуры организации программы, когда код мог состоять из нескольких изолированных модулей со своими наборами изолированных друг от друга подпрограмм и наборами данных, появилась идеология передачи параметров в подпрограммы, а следовательно, появилась возможность разделять данные. Тем не менее многие недостатки еще продолжали сохраняться, особенно когда надо было разрабатывать очень большие проекты большим коллективом разработчиков. В это время появились такие известные и до сих пор широко используемые языки, как Pascal, C, Ada.

Можно говорить о том, что это время стало периодом расцвета процедурного (структурного) программирования. Процедурное программирование рассматривает программы как набор многократно используемых единиц кода (процедур), для которых определены входные и выходные параметры. Подпрограммы могут вызывать друг друга, даже если они находятся в различных модулях проекта.

Проблемой процедурного программирования является то, что повторное использование кода во многих случаях ограничено: если меняется тип данных, надо создавать новую подпрограмму, если одни и те же данные можно обрабатывать по-новому, то встает вопрос о том, как учесть или исключить возможность предыдущей реализации. При этом еще и необходимо помнить,

как вообще можно обрабатывать те или иные данные в проекте. Иногда процедурный подход еще называют алгоритмическим, хотя вернее говорить было бы об алгоритмической декомпозиции задачи. При этом способе программирования во главу угла ставится алгоритм, который воздействует на некоторые данные. С одной стороны, это удобно, но если изменились данные, то насколько легко можно изменить алгоритм, особенно если он описывает взаимодействие многих различных типов данных?

Следующим шагом в развитии теории разработки программного обеспечения стало появление идеи абстрактной структуры данных, вершиной которой стало понятие объекта. Гради Буч [1] определяет объект как осязаемую реальность, проявляющую четко выделяемое поведение. С точки зрения восприятия человека, объектом может быть:

- осязаемый и (или) видимый предмет;
- нечто, воспринимаемое мышлением;
- нечто, на что направлена мысль или действие.

Объектно-ориентированный подход основан на парадигме, согласно которой каждая программа работает с данными, которые описывают сущности (объекты или события) из реальной жизни. Например: бухгалтерские программные системы работают со счетами, товарами, складами, наличием, заказами на продажу и т.д., системы по продажам билетов – с пассажирами, рейсами, билетами, самолетами или поездами, маршрутами движения транспорта.

Во главу угла в этом случае ставится не алгоритмическая, а объектная декомпозиция задачи. Теперь выделяют объекты, которые взаимодействуют между собой для решения той или иной задачи. Такая идеология позволяет упростить построение сложных систем, так как теперь подсистему можно рассматривать как отдельный объект, у которого есть свой способ взаимодействия с другими объектами.

Разницу между алгоритмическим и объектным взглядом на разработку программного обеспечения можно увидеть на следующем примере.

Пример. На книжной полке лежат книги. Необходимо попросить другого человека принести одну из них.

Алгоритмический вариант решения: назвать книгу, точно описать человеку, где находится полка, где на ней книга и даже как её оттуда взять: выдвинуть первые три книги слева, положить их на стол, потом взять нужную книгу, положить её на стол, потом взять три книги и вернуть их на место. Мы точно описали, что надо делать, но, если изменятся условия (например, книга стоит дальше, человеку неудобно снимать три книги вместе, стол отодвинут далеко слишком или вообще убран, или нам вообще не важно, что будет с книгами, снятыми с полки), что будет с алгоритмом?

Объектный вариант решения: назвать книгу, попросить принести книгу. Вопрос о том, как достать книгу с полки, объект решает самостоятельно, в соответствии со своей моделью поведения и своими возможностями.

Надо отметить, что объектный подход к программированию никоим образом не исключает алгоритмический! Ведь свое поведение объекты реализуют в соответствии со своими алгоритмами поведения, и их всё равно придется проектировать и реализовывать. Преимущество объектного подхода заключается в том, что если у разработчика есть набор готовых объектов с известным поведением, то он может только определять их совместное поведение, не вдаваясь в реализацию для каждого отдельного объекта. При этом если реализация поведения объекта изменится, а результат останется прежним, то никто (кроме самого объекта) этого не заметит.

Первые объектные языки программирования, такие как Simula, использовали понятие объекта, но еще не являлись в полной мере объектно-ориентированными в нынешнем понимании этого термина. Чтобы говорить об объектно-ориентированном программировании (ООП) как о методологии программирования, основанной на представлении программы в виде совокупности объектов, каждый из которых является экземпляром определенного класса, а классы образуют иерархию наследования [1], необходимо было научиться рассматривать однотипные объекты как представителей одного класса (это увеличило возможность повторяемости кода) и ввести понятие иерархии сходных или сложных объектов.

Таким образом, для того чтобы язык программирования был объектно-ориентированным, он должен уметь работать с классами и объектами, а также реализовывать и использовать следующие основные объектно-ориентированные принципы:

- абстракция;
- инкапсуляция;
- наследование;
- полиморфизм;
- модульность.

2 Основные принципы ООП

2.1 Абстракция и инкапсуляция

Понятие абстракции необходимо для понимания того, как разработчик собирается описывать объект в программе. «**Абстракция** выделяет существенные характеристики некоторого объекта, отличающие его от всех других видов объектов, и, таким образом, четко определяет его концептуальные границы с точки зрения наблюдателя. Использование абстракции концентрирует внимание на внешних особенностях объекта и позволяет отделить самые существенные особенности поведения от несущественных» [1].

Когда разработчик или аналитик начинает изучать предметную область, для которой надо разработать программу, основной проблемой становится понимание того, что надо знать об объекте для решения задачи, что является не нужным, а может быть даже и вредным. Можно сказать, что правильно построенная абстракция означает работу с чем-то, что мы знаем, как использовать, но не знаем, как это работает внутри, и не знаем об объекте ничего кроме того, что надо знать, чтобы с ним работать. Например, совершенно не надо знать, как устроен внутри телевизор, чтобы включить его, интересует только вопрос о том, где находится кнопка «вкл/выкл» и как настроить каналы и громкость. Лишние знания об объекте можно сравнить с попыткой включить телевизор при снятой задней крышке – работать будет, но вопрос, как долго и насколько безопасно для окружающих.

Возвращаясь к предыдущему примеру с телевизором надо сказать, что даже если в абстракцию и не включать информацию о внутреннем устройстве объекта, то возможность снять крышку останется все равно, поэтому хотелось бы скрыть для безопасности объекта (и окружающих тоже) детали реализации объекта, а оставить доступным только интерфейс для общения с объектом.

Инкапсуляция (скрытие) – это процесс отделения друг от друга элементов объекта, определяющих его устройство и поведение; инкапсуляция служит для того, чтобы изолировать контрактные обязательства абстракции от их реализации [1].

Одной из функций инкапсуляции является еще и разграничивание абстракций между собой. Так покупатель видит только внешний интерфейс телевизора, для мастера по ремонту абстракция расширена – он видит платы или провода, установленные в телевизоре, но при этом не видит их внутреннее устройство.

В языках программирования инкапсуляция обычно реализуется с помощью модификаторов доступа `public`, `private`, `protected`. Модификатор `public` означает, что все компоненты объекта, описанные с ним, доступны для других пользователей, т.е. они могут обращаться к объекту через такие компоненты. Модификаторы `private`, `protected` скрывают соответствующие компоненты от посторонних.

Обычно все характеристики объекта описывают в `private` части, поведение объекта по отношению к внешнему миру определяется в `public` разделе, внутреннее поведение объекта – в `private` или `protected` частях описания.

Когда мы говорим об абстракции и инкапсуляции объекта, мы должны понимать, что речь на самом деле идет не об одном объекте, а о наборе одинаковых объектов. Что подразумевается под одинаковостью объектов в этом случае? Это совпадение их поведения как внешнего, так и внутреннего, и одинаковый набор характеристик. Принято множество всех объектов, обладающих одинаковой структурой и имеющих одинаковое поведение, называть классом. Говорят, что поведение класса реализуется через его методы (их еще называют членами класса), а структура данных – через атрибуты или характеристики.

Когда разработчик проектирует абстракцию и определяет правила инкапсуляции, то он на самом деле разрабатывает класс, определяет его интерфейс (раздел `public`) и инкапсулирует его внутреннюю структуру и поведение. Затем в программе будут определены экземпляры класса или другому объект, принадлежащие классу.

Описание и использование класса, модификаторы доступа

Рассмотрим на примере использование принципов абстрагирования и инкапсуляции ООП.

Пример. Заказчик решил создать программу, моделирующую работу зоопарка. В начале было заявлено, что единственным видом животных в зоопарке являются медведи.

Разработчик должен описать класс `Bear` и задать всем медведям какое-то поведение и характеристики. Заказчик сказал, что для него сейчас важны функции кормления медведя мёдом и движения медведя, также важен вес медведя, еще у каждого медведя есть имя. То есть поведение – умение есть, умение двигаться, а атрибуты – вес и имя. Тогда первый вариант кода будет выглядеть так:

```
class Bear {
public:
    double weight;
    string name;
    Bear() {}
    void IsEating() {
        cout << "Медведь " << " " << name << " ест мед";
    }
    void Go() {
        cout << "Медведь " << " " << name << " идет";
    }
};
```

Определение класса начинается со слова `class` и заканчивается точкой с запятой, содержит атрибут `name` – имя медведя, атрибут `weigh` – вес медведя, и методы поведения `isEating()` и `Go()`, а также конструктор `Bear()`.

Каждый класс обязательно должен содержать хотя бы один конструктор, это требование является обязательным для всех языков программирования, в C++ имя конструктора совпадает с именем класса. Если у класса нет конструктора, то объект класса создать будет нельзя. В данном случае был использован так называемый пустой конструктор по умолчанию, который отвечает только за создание объекта в памяти, но не задает никаких значений для объекта. Другие варианты конструкторов будут рассмотрены ниже. В некоторых случаях в описании класса есть еще метод-деструктор, который отвечает за уничтожение объекта и очистку памяти, но так как для решения многих учебных задач можно положиться на обычные методы сборки мусора и очистки памяти, то деструкторы будут рассмотрены позднее.

Обычно принято разделять описание и реализацию класса, если это позволяют средства языка. В C++ такое возможно. Описание класса записывают в заголовочный файл "имякласса".h, а реализацию поведения – в файл "имякласса".cpp, к этому файлу директивой `#include` подключают .h-файл. Такое описание будет иметь вид:

```
// Bear.h
class Bear {
public:
    double weight;
    string name;
    Bear() {}
    void IsEating();
    void Go();
};

// Bear.cpp
#include <iostream>
#include "Bear.h"
void Bear::IsEating() {
    cout << "Медведь " << name << " ест мед\n";
}
void Bear::Go() {
    cout << "Медведь " << " " << name << " идет";
}
```

Имени метода в cpp-файле предшествует имя класса (`Bear` в данном случае) и оператор `::`, который означает, что метод `IsEating()` является членом класса `Bear`.

Когда классы небольшие по размеру и методы просты, часто весь код пишут в .h-файл, чтобы уменьшить количество файлов в проекте, но если

затем описание класса начинает увеличиваться в размере, то файл становится плохо читаемым.

В программе обращение к классу можно осуществить следующим образом:

```
Bear bear;  
bear.name = "Винни";  
bear.weight = 10;  
cout << "Медведь " << bear.name << " живет в зоопарке"  
<< '\n';  
bear.IsEating();  
bear.Go();
```

В этом примере создан экземпляр класса `Bear` (или переменная типа `Bear`) с именем `bear`. Затем, так же, как и при работе со структурами, мы обратились к полю `name`, задав ему значение "Винни", и к полю `weight`. Потом были вызваны методы кормления медведя и движения.

В результате получим:

```
Медведь Винни живет в зоопарке  
Медведь Винни ест мед  
Медведь Винни идет
```

Надо отметить, что в данном коде есть существенный недостаток – имя медведя и его вес находятся в разделе описания `public`. С точки зрения грамотной работы с классами и объектами это недопустимо, так как это означает, что любой пользователь этого класса (сотрудник зоопарка) по ошибке или специально может изменить имя медведя или задать ему отрицательный вес, хотя в его обязанности входит только кормить зверя.

Это произошло из-за того, что при описании класса был использован только модификатор доступа `public` (если его не использовать, то по умолчанию будет установлен `private`, и работать с классом будет невозможно). Поэтому разработчик должен скрыть такие атрибуты от посторонних. Это будет сделано с помощью модификатора `private`. Но тогда возникнет проблема с тем, как узнать имя и вес животного. Если предположить, что имя животного не меняется, а задается при его рождении, то тогда имя должно определяться при создании объекта (т.е. при вызове конструктора), а вот вес может меняться, поэтому можно определить начальный вес животного и способы изменения веса.

Уже на этом примере становится видно, что изменение взгляда на инкапсуляцию повлекло за собой изменение абстракции объекта – придется добавлять новые методы к классу. Обычные правила установки типа доступа и определения интерфейса для работы с атрибутами объектов следующие:

- 1) все атрибуты класса делаются закрытыми `private`;
- 2) для доступа к атрибутам, значение которых не может меняться из обращающейся к классу программы, определяется метод-селектор (или геттер), который принято начинать со слова `get`, а вторая часть имени – это

имя атрибута. Значения таких атрибутов должны задаваться либо при создании объекта в конструкторе, либо с помощью каких-либо скрытых `private` методов класса, описывающих внутреннее поведение объектов;

3) для доступа к атрибутам, значение которых может меняться из обращающейся к классу программы, определяется также метод-селектор, но еще добавляется метод-модификатор (сеттер), который принято начинать со слова `set`, а вторая часть имени – это имя атрибута. При правильной разработке модификаторов желательно проверять допустимость вводимых значений атрибута. Начальное значение атрибута лучше задавать при создании класса, если это возможно.

Изменим описание класса следующим образом: можно только узнать имя медведя и вес медведя, вес медведя может меняться только в результате кормежки, начальный вес 10 кг, и при каждой кормежке корм может быть разного веса:

```
// Bear.h
class Bear {
private:
    double weight;
    string name;
    void setWeight(double we);

public:
    Bear(string n);
    void IsEating(double we);
    void Go();
    string getName();
    double getWeight();
};

// Bear.cpp
#include <iostream>
#include "Bear.h"

Bear::Bear(string n){
    name = n;
    weight = 10;
}

void Bear::IsEating(double we){
    if (we > 0) {
        setWeigth(we);
        cout << "Медведь " << name << " съел" << we << "кг меда\n";
    }
    else cout << "Медведю " << name << " не дали мед\n";
}

void Bear::setWeigth(double we){
    weight = weight + 0.1 * we; // 10% корма на прибавку к весу
}
```

```

void Bear::Go() {
    cout << "Медведь " << " " << name << " идет\n";
}
string Bear::getName() { return name; }
double Bear::getWeight() { return weight; }

```

Тогда предыдущий пример работы с классом примет вид:

```

Bear bear("Винни");
cout << "Медведь " << bear.getName() << " живет в
зоопарке" << '\n';
bear.IsEating(3);
bear.Go();
cout << "Медведь " << bear.getName() << " весит " <<
bear.getWeight() << " кг\n";
bear.IsEating(-3); //корм отрицательного веса!

```

В результате получим:

```

Медведь Винни живет в зоопарке
Медведь Винни ест мед
Медведь Винни весит 13 кг
Медведь Винни идет
Медведю Винни не дали мед

```

Видно, что изменение уровня доступа изменило и способ работы с объектом, и если попытаться обратиться напрямую к полю `name`, например так:

```

bear.name = "Винни";

```

то будет получено сообщение об ошибке еще на стадии компиляции программы.

2.2 Иерархия, наследование и полиморфизм

Прежде чем говорить о наследовании, необходимо определить понятие иерархии с точки зрения ООП.

Так как количество объектов, а соответственно и абстракций, бесконечно, то на интуитивном уровне мы всегда пытаемся как-то обобщить или структурировать эти абстракции. Принято говорить, что:

«Иерархия» – это упорядочение абстракций, расположение их по уровням. Основными видами иерархических структур применительно к сложным системам являются структура классов (иерархия «is-a») и структура объектов (иерархия «part of»)[1].

Иерархия «is-a» или «это – разновидность того» служит для обобщения сходных по каким-то характеристикам или поведению классов. Она позволяет

выделить то, что является для них общим, и создать на основе этой общности так называемый родительский или базовый класс.

Например, мы автоматически говорим о телевизоре, утюге, стиральной машине или компьютере, что они все являются электроприборами. И, соответственно, ждем, что у всех будет кнопка «вкл/выкл», будет известна потребляемая мощность и допустимое напряжение.

Идея проектирования иерархии наследования в программе состоит в том, что разработчик определяет, какие объекты обладают каким набором общих методов или характеристик, важных для него с точки зрения задачи. Этот набор разработчик описывает как базовый класс, задав для него интерфейс взаимодействия с окружающим миром. Затем для каждого наследника можно создать свой класс, в который будут добавлены специфические для наследника атрибуты и элементы поведения и, возможно, будут изменены в реализации общие методы.

В наследственной иерархии общая часть структуры и поведения сосредоточена в наиболее общем родительском классе. По этой причине говорят о наследовании, как об иерархии «обобщение-специализация». Родительские классы при этом отражают наиболее общие, а наследники – более специализированные абстракции, в которых члены родителя могут быть дополнены, модифицированы и даже скрыты[1].

Основная проблема при проектировании иерархии – правильно выбрать общие атрибуты и методы, иначе можно получить утюг с клавиатурой или холодильник с функцией глажки.

Одиночное наследование

Пример. Внесем изменения в наш сквозной пример. В зоопарке появился еще один представитель животного мира – павлин. Если бы механизма наследования не было, то пришлось бы описывать все методы, определенные для павлина, заново, а это означает достаточно большое дублирование кода – павлинов также надо кормить, у них есть имена, и они могут двигаться. Правда, павлины еще умеют летать.

Логично будет создать базовый класс `Animalia`, в котором будут собраны общие методы и атрибуты:

```
class Animalia {
private:
    double weight;
    string name;
    void setWeight(double we);
public:
    Animalia (string n, double w); // у разных видов
    // животных разный
    // начальный вес
    void IsEating(double we, string e);
    void Go();
```

```

    string getName();
    double getWeight();
};
Animalia::Animalia (string n, double w){
    name = n;
    weight = w;
}
void Animalia::IsEating(double we, string e){
    if (we > 0) {
        setWeight(we);
        cout << name << " съел " << we << " кг корма " << e
<< "\n";
    }
    else cout << name << " не дали корм\n";
}
void Animalia::setWeight(double we){
    weight = weight + 0.1 * e; // 10% корма на прибавку
к весу
}
void Animalia::Go() {
    cout << name << " идет\n";
}
string Animalia::getName() { return name; }
double Animalia::getWeight() { return weight; }

```

Отметим, что создание базового класса немного изменило описание функции кормления животных: пришлось добавить название корма для животного, так как павлины не едят мед, да и медведи на самом деле тоже питаются не только медом. Чем отличаются медведи и павлины в поведении? Павлины умеют летать, а медведям добавим для примера умение ловить рыбу.

Тогда можно определить два класса-потомка: медведи — Bear, павлины — Peacock, и задать для них различающийся функционал.

```

class Bear : public Animalia {
public:
    Bear(string n, double w);
    double TakeFish();
};

class Peacock : public Animalia {
public:
    Peacock(string n, double w);
    void Fly();
};

```

со следующей реализацией:

```

Bear::Bear(string n, double w): Animalia(n,w){
}
double Bear::TakeFish(){
    double weightFish = random()%1000;

```

```

    cout << "Медведь " << name << " поймал рыбу весом " <<
    weightFish << " кг\n";
    return weightFish;
}
Peacock::Peacock (string n, double w): Animalia(n,w){
}
void Peacock:: Fly(){
    cout << "Павлин " << name << " летит " <<"\n";
}

```

При описании наследников после имени класса-потомка ставится :<модификатор доступа> <имя родительского класса>. Эта конструкция определяет тип наследования – public, private – и от какого базового класса оно выполнено. Если указано public-наследование, то все public атрибуты и методы базового класса будут иметь в классе-потомке уровень доступа public, если указано private-наследование, то все public методы и атрибуты базового класса попадают в private раздел дочернего класса и могут быть доступны только из него.

Поля и атрибуты, отмеченные как private в родительском классе, останутся невидимыми для класса-потомка, и доступ к ним будет возможен только через геттеры и сеттеры базового класса.

Особое место занимает модификатор protected в описании базового класса: те атрибуты и методы, которые были описаны в разделе protected, будут доступны для методов класса-потомка так же, как если бы они находились в разделе public базового класса.

Чаще всего private-наследование используют в тех случаях, когда надо полностью скрыть устройство базового класса от классов-потомков, создать что-то подобное черному ящику – функционал использовать можно, а доступ к внутренней структуре получить нельзя извне.

Теперь создадим объекты этих классов и посмотрим на работу с ними:

```

Bear bear("Винни",100);
cout << "Медведь " << bear.getName() << " живет в
зоопарке" << '\n';
bear.IsEating(3,"Мед");
bear.Go();
Peacock peacock("Павлуша", 10);
    cout << "Павлин " << peacock.getName() << " живет в
зоопарке" << '\n';
peacock.IsEating(3, "Трава");
peacock.Go();
peacock.Fly();

```

получим следующий результат:

```

Медведь Винни живет в зоопарке
Винни съел 3 кг корма Мед
Винни идет

```

```
Павлин Павлуша живет в зоопарке
Павлуша съел 3 кг корма Трава
Павлуша идет
Павлин Павлуша летит
```

Теперь появляется возможность использовать какой-то общий метод для всех животных зоопарка, например, надо узнать вес и имена всех животных и перевести в другое место. Если бы класса-предка не было, то пришлось бы узнавать вес и имена павлинов и медведей разными фрагментами кода, а в данном случае достаточно написать следующее:

```
Bear bear("Винни",100);
Peacock peacock("Павлуша", 10);
Bear bear1("Пух", 200);
list<Animalia*> Zoo;      //список всех животных зоопарка
Zoo.push_back(&bear);    //добавляем в него живущих
Zoo.push_back(&peacock); //сейчас в зоопарке
for each (Animalia* var in Zoo)
{
    var->Go();
    cout << var->getName() << " весит " << var->getWeight()
    << " кг\n";
}
```

и тогда можно получить полную информацию о животных зоопарка:

```
Винни идет
Винни весит 100.3 кг
Павлуша идет
Павлуша весит 10.3 кг
```

Если бы все преимущества наследования ограничивались бы только возможностью повторного использования кода и возможностью определения одинакового поведения для всех родственных объектов, то ООП не приобрело бы такой популярности среди разработчиков.

Наследование позволяет, с одной стороны, обращаться с объектами-наследниками, как с объектами базового класса, с другой стороны, реализовывать эти действия по-своему.

В классах-потомках возможно переопределение общего функционала класса, если это необходимо.

Пример. Изменим реализацию метода `Go()` для павлина. Павлин будет двигаться вприпрыжку, а не шагом.

Тогда в описание класса `Peacock` добавится метод `Go` со следующей реализацией:

```
void Peacock::Go() {
    cout << "Павлин " << " " << getName() << "
подпрыгивает на ходу\n";
}
```

и тот же код, что и в предыдущих двух примерах даст другой результат:

```
Медведь Винни живет в зоопарке
Винни съело 3 кг корма мед
Винни идет
Павлин Павлуша живет в зоопарке
Павлуша съел 3 кг корма Трава
Павлуша подпрыгивает на ходу
Павлин Павлуша летит
Винни идет
Винни весит 100.3 кг
Павлуша идет
Павлуша весит 10.3 кг
```

Видно, что когда обращение было напрямую к объекту класса `Peacock`, то двигаться павлин стал по-другому, а вот когда обратились к нему, как к разновидности животных, то движение стало прежним.

Произошло так называемое скрывание метода дочернего класса при обращении через родительский класс. Имена совпадают, а тип объекта определяется типом объекта, через который происходит обращение, в данном случае это тип `Animalia`, поэтому были «потеряны» все индивидуальные особенности класса `Peacock`, они заменились общими методами.

Чтобы исправить эту ситуацию, необходимо сообщить компилятору, что надо проверять реальный тип объекта при обращении к его методам. Для этого те методы, которые хотим вызывать для сохранения специфики объекта, надо объявлять с ключевым словом `virtual` в родительском классе:

```
class Animalia {
...
public:
    ...
    virtual void Go();
...
};
```

В этом случае, при компиляции будет определен реальный тип объекта и будет выполнена его реализация метода:

```
Медведь Винни живет в зоопарке
Винни съел 3 кг корма мед
Винни идет
Павлин Павлуша живет в зоопарке
Павлуша съел 3 кг корма Трава
Павлин Павлуша подпрыгивает на ходу
Павлин Павлуша летит
Винни идет
Винни весит 100.3 кг
Павлуша подпрыгивает на ходу
```

Павлуша весит 10.3 кг

Обычно ключевое слово `virtual` добавляют сразу и для методов дочернего класса, на случай, если придется создавать более разветвленную иерархию наследования, чтобы в дальнейшем не забыть об этом. Методы, описанные со словом `virtual`, называются **виртуальными**.

Возможность использовать для объектов с одинаковым описанием различные реализации называется **полиморфизмом**. Именно полиморфизм позволяет создавать гибкий код, который будет работать с каждым объектом в программе так, как необходимо этому объекту.

Если по каким-то причинам надо обратиться из потомка к методу именно родительского класса, то перед вызовом метода указывается имя класса:

```
peacock.Animalia::Go();
```

При проектировании иерархии наследования очень часто разработчики сталкиваются с ситуацией, когда для родительского класса можно сформулировать интерфейс, указав все необходимые методы, но нельзя сказать, как они будут реализовываться. Например, если создать иерархию геометрических фигур, то понятно, что должен быть метод вычисления площади фигуры, но определить, как это делается для произвольной фигуры, невозможно.

Одним из вариантов решения проблемы является создание пустых методов или методов, возвращающих ноль:

```
void Figure::Print() {}  
double Figure::Area() { return 0; }
```

Но это неправильный путь, который может привести к большим ошибкам в программе, если кто-то создаст объект родительского класса, рассчитает площадь, а потом разделит (или, еще хуже, умножит) на неё другое число. Поэтому в объектно-ориентированных языках программирования введены понятия чисто виртуального метода и абстрактного класса.

Метод является чисто виртуальным, если имя метода приравнивается к нулю:

```
class Animalia l {  
    public:  
        ...  
        virtual void IsEating(string e, double we) = 0;  
        virtual void Go() = 0;  
        ...  
};
```

В этом случае нельзя будет задать реализацию метода в классе и, значит, нельзя будет создать экземпляр этого класса. Так делают в тех случаях, когда родительский класс является настолько общим, что создание экземпляра этого класса не имеет смысла.

Классы, содержащие чисто виртуальные методы, называются **абстрактными классами**.

Все классы-наследники абстрактного класса должны переопределить у себя чисто виртуальный метод, если метод останется не переопределенным, то соответствующий класс также будет абстрактным.

Множественное наследование

Язык C++ – один из немногих языков программирования, поддерживающих множественное наследование. Множественное наследование подразумевает, что у класса-потомка может быть несколько родителей. Такое возможно, если с точки зрения разработчика проектируемый класс в равной степени обладает поведением двух или более несвязанных друг с другом классов.

Пример. Есть класс Car, описывающий машины. Для каждой машины заданы ее марка и скорость движения:

```
class Car {
private:
    string marka;
    int curSpeed;
public:
    Car(string m);
    int getSpeed();
    void setSpeed(int sp);
};
```

и класс Emergency для описания транспорта аварийных служб:

```
class Emergency {
private:
    int status;
    string Color;
protected:
    virtual Go();
public:
    Emergency();
    void Signal();
    void NoAlarm();
    void Alarm();
};
```

Известно, что такой транспорт имеет особую раскраску, звуковой сигнал, особые правила движения по тревоге. Тогда логичным кажется создать машину скорой помощи как потомка этих двух классов, так как они обладают абсолютно разным поведением:

```
class AmbulanceCar : public Car, public Emergency {
public:
```

```
AmbulanceCar(string m);  
};
```

В принципе такая реализация удобна. Если надо будет подать сигнал всему аварийному транспорту, то экземпляр класса `AmbulanceCar` также сможет включить сирену, а если необходимо задать скорость движения машин, то это можно сделать обращением к методу, унаследованному от `Car`.

Почему же множественное наследование используется достаточно редко, а многие языки его не поддерживают вовсе?

Во-первых, для правильной работы такого наследования необходима полная независимость классов-родителей друг от друга, а во-вторых, если случайно у родителей совпадут названия методов, то программа не пройдет даже компиляцию.

Например, если для класса `Car` будет определен метод `Go()`, то возникнет конфликт имен – в вызове

```
AmbulanceCar ac("Volvo");  
ac.Go();
```

к какому методу обращаемся? Унаследованному от `Car` или от `Emergency`? Необходимо явно указать, к методу какого класса идет обращение:

```
ac.Emergency::Go();  
ac.Car::Go();
```

Такая же проблема возможна и с именами атрибутов. Второй проблемой множественного наследования является случай ромбовидного наследования, когда класс-потомок имеет нескольких родителей, а вот у них есть один общий предок. В этом случае все атрибуты и методы общего предка будут включаться в потомка столько раз, сколько раз предок встречается в описании родителей. Единственным способом избежать конфликта является добавление к типу наследования (`public` или `private`) ключевого слова `virtual`:

```
class Base {  
    int Id;  
public:  
    Base();  
};  
class Derived1 : virtual public Base {  
    int Id_d1;  
public:  
    Derived1();  
};  
class Derived2 : virtual public base {  
    int Id_d2;  
public:  
    Derived2();  
};
```

```
};
// Derived3 является потомком как Derived1, так и
// Derived2. В Derived3 одна копия класса Base.
class Derived3: public Derived1, public Derived2 {
    int Id_d3;
public:
    Derived3();
};
```

Споры об использовании множественного наследования продолжаются много лет. С одной стороны, это повышает гибкость кода, с другой – может способствовать возникновению ошибок даже в ранее работавшем коде, если вдруг у одного из базовых классов появятся методы или атрибуты с именами, совпадающими с именами из другого базового класса. Это вполне возможно, так как базовые классы "не подозревают" о существовании друг друга. Поэтому многие проектировщики придерживаются мнения, что множественное наследование может быть заменено на сочетание одиночного наследования и агрегирования.

Агрегирование

Когда мы определяли понятие иерархии, то говорили о том, что есть иерархия вида "это – часть того". Такая иерархия и определяет агрегирование объектов. Агрегирование используется в тех случаях, когда объект состоит из набора других объектов: самолет собирается из фюзеляжа, шасси, крыльев, двигателей, рулей высоты и т.д.; государство с точки зрения административной структуры может состоять из округов, округа включают в себя области, области – районы и т.д., до уровня населенных пунктов.

Агрегирование может заменить множественное наследование. Например, машину скорой помощи можно описать тогда как машину, имеющую дополнительный функционал и атрибуты, связанные с аварийными службами.

```
class AmbulanceCar : public Car {
    Emergency* em;
public:
    AmbulanceCar(string m);
};
```

В этом случае машина будет работать как обычная машина, во всех случаях, когда она не задействована в аварийных работах.

Агрегирование является мощным средством проектирования программ, так как позволяет собирать объекты из уже готовых частей, имеющих полностью описанный функционал, задача агрегации в этом случае – описать совместную работу всех составных частей.

Агрегировать объекты можно двумя способами: использование (собственно агрегация) и владение (композиция). Если один объект использует другой, то это означает, что объемлющий объект работает со своей

частью, но часть может существовать без целого. Например, сотрудник может существовать без организации (он перейдет в другую), если организация будет закрыта. В случае владения часть не может существовать без целого – отдел есть, пока есть организация. Чтобы продемонстрировать разницу между этими способами, рассмотрим следующий пример.

Пример. В организации есть несколько отделов, отдел задается своим названием, в отделе есть сотрудники, о них известно имя и должность, сотрудник принимается в конкретный отдел, перед расформированием фирмы или отдела сотрудники должны быть уволены.

```
// класс служащих
class Employer {
    string name;
    string position;
public:
    Employer(string n, string p) {
        name = n;
        position = p;
    };
// узнать имя сотрудника
    string getName() { return name; };
// узнать должность сотрудника
    string getPosition() {return position;};
// задать должность сотрудника
    void setPosition(string p) {position = p;};
};

//класс отделов
class Department {
    string name;
    list<Employer*> lstEmployer;// список сотрудников
public:
    Department(string n) {
        name = n;
    };
    string getName() { return name; }
    //принять служащего в отдел
    void AddEmployer(Employer* emp);
    // уволить служащего из отдела
    void RemoveEmployer(Employer* emp);
    // Расформировать отдел
    void DelDepartment();
    // Напечатать состав отдела
    void PrintDep();
};

void Department::AddEmployer(Employer* emp) {
    lstEmployer.push_back(emp);
}

void Department::RemoveEmployer(Employer* emp) {
    lstEmployer.remove(emp);
}
```

```

// Расформировать отдел
void Department::DelDepartment() {
// предварительно уволив сотрудников -
for each (Employer* var in lst_Employer)
{
    // все сотрудники отдела теперь ищут работу
    var->setPosition("ищет работу");
}
lst_Employer.clear();
}

void Department::PrintDep() {
cout << "В отделе " << name << " работают: ";
for each (Employer* var in lst_Employer)
{
    cout<< var->getName() << " ";
}
cout << '\n';
};

// класс фирм
class Firma {
private:
    string name;
    list<Department*> lst_Department;//список отделов
public:
    Firma(string n) {
        name = n;
    };
    // Создать и удалить отдел
    void AddDep(string nDep);
    void RemoveDep(string nDep);
    // Принять и уволить служащих
    void AddEmpl(Employer* emp, string nDep);
    void RemoveEmpl(Employer* emp, string nDep);
    // Расформировать фирму
    void DelFirma();
    void PrintFirma();
};

void Firma::AddDep(string nDep) {
    lst_Department.push_back(new Department(nDep));
}

void Firma::RemoveDep(string nDep) {
    list<Department*>::iterator it;
//нашли отдел, который расформировывается
    for ( it = lst_Department.begin();
        it != lst_Department.end(); it++)
    {
        if ((*it)->getName() == nDep) break;
    };
    // удалили его из фирмы
    it->DelDepartment();
    lst_Department.remove(*it);
    delete (*it);
};

```

```

void Firma::AddEmpl(Employer* emp, string nDep) {
    list<Department*>::iterator it;
    for (it = lstDepartment.begin();
         it != lstDepartment.end(); it++)
    {
        if ((*it)->getName() == nDep) break;
    };
    if (it != lstDepartment.end())
        (*it)->AddEmployer(emp);
}

void Firma::RemoveEmpl(Employer* emp, string nDep) {
    list<Department*>::iterator it;
    for (it = lstDepartment.begin();
         it != lstDepartment.end(); it++)
    {
        if ((*it)->getName() == nDep) break;
    };
    if (it != lstDepartment.end())
        (*it)->RemoveEmployer(emp);
};

void Firma::DelFirma() {
    // предварительно расформировав отделы
    for each (Department* var in lstDepartment)
    {
        var->DelDepartment();
    }
    lstDepartment.clear();
}

void Firma::PrintFirma() {
    cout << "Структура фирмы " << name << ": \n";
    for each (Department* var in lstDepartment)
    {
        var->PrintDep();
    }
    cout << '\n';
};

```

В примере видна разница между способами создания и удаления для сотрудников и отделов. На работу принимается в методе AddEmpl уже существующий объект – сотрудник, и когда сотрудник увольняется, то из списка сотрудников просто удаляется ссылка на этот объект, а сам объект остается неизменным. В то же время объект отдела создается внутри фирмы методом AddDep, который задает только имя отдела, и при удалении из списка отделов уничтожается. Когда фирма прекратит свое существование, то отделы будут расформированы, а вот сотрудники (т.е. экземпляры класса Employer)смогут устроиться в другую фирму. И, кстати, сотрудники могут одновременно работать в нескольких организациях на одной и той же должности!

Создадим две фирмы и четырех служащих:

```

Firma f1("Рога и копыта");
Firma f2("Шерсть и перья");
Employer emp1("Иванов", "Директор");
Employer emp2("Петров", "Бухгалтер");
Employer emp3("Сидоров", "Кассир");
Employer emp4("Кузнецов", "Инженер");

```

В фирму «Рога и копыта» добавим отдел «Администрация» и примем туда на работу Иванова и Петрова.

```

f1.AddDep("Администрация");
f1.AddEmpl(&emp1, "Администрация");
f1.AddEmpl(&emp2, "Администрация");
f1.PrintFirma();

```

В фирму «Шерсть и перья» добавим отдел «Бухгалтерия» и примем туда на работу Сидорова и Кузнецова.

```

f2.AddDep("Бухгалтерия");
f2.AddEmpl(&emp3, "Бухгалтерия");
f2.AddEmpl(&emp4, "Бухгалтерия");
f2.PrintFirma();

```

Создадим в фирме «Рога и копыта» отдел «Касса» и примем туда по совместительству кассира Сидорова.

```

f1.AddDep("Касса");
f1.AddEmpl(&emp3, "Касса");

```

Теперь попробуем удалить фирму "Шерсть и перья" и принять Кузнецова в «Рога и копыта».

```

f2.DelFirma();
f1.AddEmpl(&emp4, "Администрация");
f1.PrintFirma();

```

Результат работы представлен ниже:

```

Структура фирмы Рога и копыта:
В отделе Администрация работают: Иванов Петров

```

```

Структура фирмы Шерсть и перья:
В отделе Бухгалтерия работают: Сидоров Кузнецов

```

```

Структура фирмы Рога и копыта:
В отделе Администрация работают: Иванов Петров Кузнецов
В отделе Касса работают: Сидоров

```

Можно сказать, что при выборе типа иерархии для классов при проектировании приложений разработчик должен понимать, с чем он имеет

дело в каждом случае. Если задача предполагает обобщение функционала различных объектов, то выбирается наследование «это – разновидность того», если же предполагается конструирование сложных систем, необходима иерархия «это – часть того». Естественно, что в реальных проектах обе иерархии постоянно используются вместе.

2.3 Конструкторы, деструкторы, дружественные функции, статические члены класса

Конструкторы

Среди методов, обеспечивающих работу с классами, особое место занимают конструкторы. Первая и самая важная их функция – это создание объекта перед началом работы с ним. Поэтому класс должен иметь хотя бы один конструктор.

На самом деле, конструкторов обычно бывает несколько:

1) Конструктор по умолчанию. Это конструктор без параметров, который задает некоторые начальные значения для полей класса так, чтобы с объектом, созданным этим конструктором, можно было начать работу. Без конструктора по умолчанию невозможно описание массивов объектов одного класса.

```
Bear::Bear () {  
    name = "";  
    weight = 0;  
}
```

В данном случае, все создающиеся медведи будут иметь нулевой вес и пустую строку в качестве имени. Предполагается, что далее эти значения будут изменены.

```
Bear bears [10]; // вызов конструктора по умолчанию –  
    // должен быть без параметров  
Bear* pbears;  
pbears = new Bear [10]; // вызов конструктора по  
    // умолчанию
```

2) Конструктор с параметрами. Такой конструктор позволяет создавать объекты с необходимым набором значений атрибутов для каждого объекта. Таких конструкторов может быть несколько, в зависимости от целей разработчика класса.

```
// задаются имя и вес медведя  
Bear::Bear(string n, double w){ name = n;  
    weight = w;  
}  
// задается имя медведя, вес у всех одинаковый – 10 кг  
Bear::Bear(string n){  
    name = n;
```

```

weight = 10;
}
// задается вес медведя, всех медведей зовут "Мишка"
Bear::Bear(double w){
name = "Мишка";
weight = w;
}

```

3) Конструктор копирования. Очень важный конструктор, без которого невозможна нормальная работа с объектами. Когда разработчик использует фундаментальные типы данных (целые, вещественные и т.д.), то операция присваивания понимается и выполняется однозначно – в переменную слева от знака = копируется значение справа от =.

```

int a,b;
a = 5;
b = a;

```

Переменная `b` получит значение 5, если будем изменять `a`, то на `b` это не отразится. Когда же идет работа с указателями, всё становится сложнее:

```

int *a,*b;
a = new int;
*a = 5;
b = a;

```

Переменная `b` будет указывать на ту же область памяти, что и `a`, и если будем изменять `*a`, то `*b` так же будет изменять значение. Мы получили не два разных объекта, а один с двумя именами. Если написать

```

*b = *a;

```

то ошибка будет исправлена. Но такой простой способ не подходит для классов, так как атрибуты класса в свою очередь могут быть объектами других классов, созданными с помощью указателей. В этом случае разименование указателя всё равно приведет к ошибке, а к тому же при присваивании происходит между объектами в целом, без использования информации о внутренней структуре объекта. Предположим, что фирму `f2` из предыдущего раздела решили создать как полную копию `f1`, по сути сделать своеобразный клон (часто используемый прием, когда исходный объект "портить" нельзя, а проверить какие-то задачи надо):

```

f2 = f1;

```

Операция будет выполнена, а вот результат будет несколько неожиданным – на самом деле списки отделов и списки сотрудников в отделах будут общими и выполнение инструкции

```
f2->RemoveEmpl ("Иванов");
```

приведет к увольнению директора из f1 тоже. Это произошло из-за того, что списки хранят указатели на объекты-сотрудники, о чем программист, использующий класс фирма, знать в принципе не обязан, так как реализация для него скрыта.

Если разработчик хочет обеспечить надежную работу со всеми объектами в своем проекте, то конструктор копирования должен быть создан для каждого класса в проекте. Общий вид конструктора копирования следующий:

```
ИмяКласса :: ИмяКласса (ИмяКласса& obj) {  
    АтрибутКласса1 = obj.АтрибутКласса1;  
    // перечисление всех атрибутов класса  
    АтрибутКласса2 = obj.АтрибутКласса2;  
    // для атрибутов-ссылок . заменяется на ->  
    АтрибутКласса3 = obj->АтрибутКласса3;  
    ...  
}
```

В этом случае каждая операция присваивания будет выполняться следующим образом: если то, что стоит справа от оператора присваивания, обладает своим конструктором копирования, то будет вызван этот конструктор, если конструктора нет, то присваивание будет идти по обычным правилам языка.

```
Employer :: Employer (Employer& emp){  
    name = emp.name;  
    position = emp.position;  
}  
Department :: Department (Department& dep){  
    name = dep.name;  
    foreach (Employer emp in dep.lstEmployer)  
        lstEmployer.push_back (emp);  
}  
Firma :: Firma (Firma& fm){  
    name = fm.name;  
    foreach (Department dep in fm.lstDepartment)  
        lstDepartment.push_back (fm.dep);  
}
```

Мы получим иерархию вызовов конструкторов копирования, и в этом случае присваивание

```
f2 = f1;
```

будет выполнено правильно.

Деструкторы

Метод деструктор отвечает за правильное уничтожение объекта класса после того, как объект перестает быть нужным. Это значит, что деструктор

должен очистить память, которую занимал объект, освободить все ресурсы, которые были захвачены объектом во время работы. Можно, конечно положиться на то, что программа сама выполнит эти задачи перед своим окончанием, но, во-первых, программа может работать сутками или вообще месяцами, и тогда мусор забьет всю память, а, во-вторых, принудительное освобождение ресурсов может привести к потере данных, например в случае работы с файлами (запись в файл) или какими-то другими внешними ресурсами.

Деструктор нельзя вызвать явно. Для объектов, созданных как динамические с помощью команды `new`,

```
Department* dep = new Department();
```

деструктор выполнится в момент вызова команды `delete`, а для объектов, созданных статически, деструктор выполнится, когда программа выйдет из области видимости этого объекта.

У деструкторов есть еще одна дополнительная возможность, очень полезная при работе с абстрактными классами: если сложно или невозможно определить, какой метод сделать чисто виртуальным, то так можно поступить с деструктором. Более того, рекомендуется делать деструктор виртуальным всегда, если от предполагается, что в дальнейшем на основе класса будет создаваться иерерхия [2]. Так же, как и конструктор, деструктор должен быть описан в разделе `public` описания класса.

Дружественные функции

В некоторых случаях разработчику необходимо предоставить доступ к полям класса функциям, которые к классу никакого отношения не имеют (один из самых часто встречающихся случаев – перегрузка операций ввода-вывода). Для этого в языке C++ предусмотрены дружественные функции. Они описываются в разделе `public` класса со спецификатором `friend`.

```
class Employer {
...
public:
...
friend ifstream& operator >>(ifstream& is, Employer&
emp);
friend ofstream& operator <<(ofstream& os, Employer
emp);
}
ifstream& operator >>(ifstream& is, Employer& emp){
is >> emp.name >> emp.position;
return is;
}
ofstream& operator <<(ofstream& os, Employer emp){
os << emp.name << " " << emp.position;
return os;
}
```

```
// Пример обращения к дружественным функциям.
ifstream in("data.txt");
ofstream out("res.txt");
Employer empl;
in >> empl;
out << empl;
```

Насколько оправдано использование дружественных функций? Многие языки программирования в принципе не поддерживают этот механизм, так как помимо удобства он создает и определенный риск. Дружественная функция получает полный доступ к классу, следовательно, если изменится её реализация, то данные могут быть испорчены.

Статические члены класса

Когда создается экземпляр класса, то каждый экземпляр имеет свой набор значений характеристик и методов. Однако, практически все объектно-ориентированные языки программирования предусматривают еще один тип членов класса – статические поля и методы класса.

В коде они описываются со словом `static` перед указанием типа поля или типа возвращаемого значения метода класса. Их смысл в том, что статический член класса один для всех создающихся в программе экземпляров класса.

Например, в класс `Animalia` можно добавить статическое поле `countAnimalia`, в котором будет храниться количество созданных экземпляров класса. При создании очередного животного количество будет увеличиваться, а при вызове деструктора – уменьшаться.

Тогда описание класса `Animalia` примет вид:

```
class Animalia {
    static int countAnimalia;
    string name;
    double weight;
public:
    Animalia () {
        countAnimalia++;
    }
    Animalia (string n, double w) {
        name = n;
        weight = w;
        countAnimalia++;
    }
    ....
    ~Animalia () {
        countAnimalia--;
    }
}
```

Инициализация статической переменной происходит вне класса:

```
int Animalia::countAnimalia = 0;
```

Одним из примеров частого использования статических переменных является случай ограничения доступа к какому-нибудь ресурсу. Например, все служащие имеют право печатать на принтере, но он один и задания можно отправлять на печать только по очереди. Тогда имеет смысл завести статическую переменную `printerLock`, которая будет отражать статус занятости принтера и будет видна всем служащим.

```
class Employer {
    static bool printerLock;
    ...
public:
    ...
    void printDoc (string nameDoc);
}
void Employer :: printDoc(string nameDoc) {
    if (!printerLock) {
        printerLock = true;
        // печать документа
    }
    else cout << "Принтер занят!\n";
}
```

Если статическое поле описано в разделе `public` класса, то к нему можно обратиться из текста основной программы, если же такое поле описано в разделе `private` (что более логично), то возникает вопрос, как обратиться к такому полю? В принципе язык поддерживает возможность обращения через обычные методы класса, но это не очень верно с точки зрения концепции инкапсуляции и защиты данных, поэтому для работы с такими полями обычно используют методы класса, которые также описываются со словом `static`.

```
class Animalia {
    static int countAnimalia;
    ...
public:
    static getCount(){return countAniamlia;}
    ....
}
int main(){
    Bear bear("Винни",100);
    Peacock peacock("Павлуша", 10);
    Bear bear1("Пух", 200);
    cout << "Всего животных "
         << Animalia::getCount() <<'\\n';
}
```

Необходимо помнить, что из статических функций нельзя обращаться к нестатическим полям класса.

2.4 Преобразование типов

Преобразование или приведение типов для объектно-ориентированного программирования является одним из инструментов реализации наследования и полиморфизма. С одной стороны, это динамическое преобразование типов (или динамическая типизация) при обращении к объектам одного класса как к объектам другого родственного класса:

```
Peacock peacock("Павлуша", 10);
Animalia* a1 = & peacock;
a1->Go();
cout << a1->getName() << " весит " << a1->getWeight()
    << " кг\n";
Павлуша подпрыгивает на ходу
Павлуша весит 10.3 кг
```

С другой стороны, в некоторых случаях может понадобиться явное преобразование объекта родительского класса к объекту дочернего класса. Естественно, это не всегда возможно, поэтому в C++ существует преобразование `dynamic_cast`, которое используется именно для безопасного преобразования по иерархии наследования. Рассмотрим пример с зоопарком:

```
Bear bear("Винни", 100);
Peacock peacock("Павлуша", 10);
Bear bear1("Пух", 200);

list<Animalia*> Zoo; //список всех животных зоопарка
Zoo.push_back(&bear); //добавляем в него живущих
Zoo.push_back(&peacock); //сейчас в зоопарке
```

Цикл, в котором животные зоопарка двигаются, не требует преобразования типов, так как функция движения существует для всех.

```
for each (Animalia* var in Zoo) var->Go();
```

Если же потребовать, чтобы выполнялись функции, специфические для каждого класса (медведь ловит рыбу, а павлин летит), то методы `TakeFish` и `Fly` будут недоступны для переменной `var`. И даже если к ним обратиться, то во время компиляции возникнет ошибка, так как при полиморфизме вызывается переопределенная общая функция, а не специфическая функция класса. Поэтому воспользуемся преобразованием типов следующим образом:

```
for each (Animalia* var in Zoo)
{
```

```

var->Go();
Bear *b = dynamic_cast<Bear*>(var);
if (b != NULL)
    b->TakeFish();
else {
    Peacock *p = dynamic_cast<Peacock*>(var);
    if (p != NULL)
        p->Fly();
}
}

```

Если удастся осуществить необходимое преобразование, то значением соответствующего указателя (b или p) будет адрес в памяти объекта соответствующего класса, и, следовательно, можно будет обратиться к нужному методу. Если преобразование будет безуспешным, то указатель получит значение NULL.

Существует еще один интересный способ узнать, с объектом какого класса сейчас идет работа в программе. Если подключить к файлу программы заголовочный файл <typeid.h>, то можно будет воспользоваться оператором typeid, который возвращает фактический тип объекта (по сути имя класса):

```

string clName;
for each (Animalia* var in Zoo)
{
    clName = typeid(var).name(); // узнали имя класса
    cout<< clName <<"\n";      // напечатали
    var->Go();
    Bear *b = dynamic_cast<Bear*>(var);
    if (clName == "class Bear")
        ((Bear)var)->TakeFish();
    else {
        if (clName == "class Peacock")
            ((Peacock)var)->Fly();
    }
}

```

Конструкция несколько более громоздкая, но, если для каких-то целей необходимо имя класса, typeid является единственным способом получить его.

Заключение

В первой части данного пособия рассмотрены основные принципы объектно-ориентированного программирования с использованием языка C++. На примерах показаны проблемы, возникающие при работе с объектами.

Пособие содержит примерные темы предметных областей для выполнения практических и лабораторных работ по проектированию собственных иерархий классов.

Список использованных источников

1. Буч Г. Объектно-ориентированный анализ и проектирование с примерами приложений // 3-е издание. – Издательский дом «Вильямс», 2017. – 720с.
2. Шилдт Г. С++. Базовый курс // 3-е издание. – Издательский дом «Вильямс», 2015. – 624с.

Приложение

Приложение А. Построение абстракций классов

Создать класс, отражающий структуру данных, согласно выбранному варианту задания.

Класс должен содержать методы для чтения, установки и отображения своих данных. Продумать, какими атрибутами (не менее 5) должна обладать структура, чтобы максимально совпадать с реальным аналогом. Продумать, какими методами должен обладать класс для максимального совпадения с реальным аналогом (не менее 5). Реализовать класс-список для управления динамической структурой из элементов созданного класса, необходимы возможности – добавления, удаления, вставки, поиска в списке, выполнения заданной операции над каждым элементом списка.

- 1) Время (время не повторяется)
- 2) Товар ((нет товаров с одинаковым артикулом))
- 3) Книга (нет одноименных книг)
- 4) Город (нет одноименных городов)
- 5) Адрес (адрес не может встретиться дважды)
- 6) Студент (не существует студентов с одинаковыми номерами зачеток)
- 7) Автомобиль (не может быть одинаковых номеров на двигателях)
- 8) Компьютер (не может быть одинаковых MAC-адресов)
- 9) Предприятие (не может быть одинакового названия и номера счета)
- 10) Страна (у двух стран не может быть столица в одном месте)
- 11) Предметный указатель
- 12) Каталог библиотеки
- 13) Расписание занятий (нельзя занимать уже занятую аудиторию)
- 14) Расписание приема пациентов (нельзя принимать пациентов одновременно одним врачом)
- 15) Множество (определить необходимые операции)
- 16) Целое число в заданной системе счисления (число должно храниться в виде массива целых чисел (разрядов числа), необходимо определить для организации ввода-вывода, операции сложения, вычитания, умножения, деления, взятия остатка от деления двух чисел и операции сравнения.)
- 17) Рациональное число (должен реализовывать стандартные операции над числами: сложение, вычитание, умножение, деление и операции сравнения. В классе также необходимо предусмотреть средства приведения дроби к смешанному виду. Рациональное число (лат. ratio – отношение, деление, дробь) – число, представляемое обыкновенной дробью n/m , где m, n – целые числа. Правильной называется дробь, у которой модуль числителя меньше модуля знаменателя. Правильные дроби представляют рациональные числа, принадлежащие интервалу $(-1, 1)$. Дробь, не являющаяся правильной, называется неправильной. У нее модуль числителя больше или равен модулю знаменателя. Неправильную

дробь можно представить в виде суммы целого числа и правильной дроби. Такая запись числа называется смешанной дробью.)

- 18) Комплексное число (с перегрузкой арифметических операций и операций сравнения)
- 19) Комплексное число в тригонометрической форме (с перегрузкой арифметических операций и операций сравнения)

Приложение В. Задания по проектированию иерархии классов

Постановка задачи: Создать набор классов согласно выбранному варианту задания. Реализовать процедуры ввода и отображения данных. Написать программу, в которой создаются объекты различных классов из иерархии и помещаются в список, после чего список просматривается. Сделать соответствующие методы (связанные наследованием) не виртуальными и посмотреть, как поведут себя экземпляры классов в этом случае.

- 1) Создать базовый класс «Фигура» и несколько производных от него классов: «Точка», «Квадрат», «Круг» и т.д. Классы должны содержать приватные, защищенные и общедоступные члены данных. Ввести функции подсчета площади и периметра геометрических фигур.
- 2) Создать базовый класс «Транспорт» и несколько производных от него классов: «Автомобиль», «Самолет», «Поезд» и т.д. Классы должны содержать приватные, защищенные и общедоступные члены данных.
- 3) Создать базовый класс «Фигура» и производные от него классы «Точка», «Квадрат», «Круг». Создать базовый класс «Оформление» и производные от него классы «Цвет» и «Стиль». На основе этих классов создать набор различных классов с использованием множественного наследования.
- 4) Создать базовый класс «Транспорт» и производные от него классы «Автомобиль», «Поезд», «Самолет». Создать базовый класс «Тип» и производные от него классы «Грузовой» и «Пассажирский». На основе этих классов создать набор различных классов с использованием множественного наследования.
- 5) Создать базовый класс «Трехмерная Фигура» и несколько производных от него классов: «Точка», «Многогранник», «Шар» и т.д. Классы должны содержать приватные, защищенные и общедоступные члены данных. Ввести функции подсчета площади и объема геометрических фигур.
- 6) Создать иерархию классов «Вагоны пассажирского поезда» с разделением на купейные, плацкартные, СВ. Каждый класс вагона должен содержать информацию о количестве мест разных типов (нижнее, верхнее, нижнее боковое, верхнее боковое), о наличии дополнительных услуг и ценах на них. С помощью виртуальных функций получить полный доход от эксплуатации вагона. Создать класс «Пассажирский поезд», который хранит список вагонов. Подсчитать доход от одного рейса поезда.

- 7) Создать иерархию классов, имитирующих поведение двух игровых персонажей разных игровых классов/рас/профессий (например, «воин» и «маг»). Каждый персонаж должен обладать набором некоторых характеристик (например, «очки здоровья», «сила атаки», «коэффициент брони», модификатор спас-бросков по ловкости, интеллекту и т.п.). Подключите разработанные классы к основной программе и реализуйте «дуэль» между этими персонажами.
- 8) Создать иерархию классов – шахматные фигуры – использовать только вертикальное наследование
- 9) Создать иерархию классов – шахматные фигуры – максимально использовать множественное наследование.
- 10) Создать иерархию классов рыб
- 11) Создать иерархию – организация, страховая компания, судостроительная компания, завод
- 12) Создать иерархию классов – журнал, книга, печатное издание, учебник
- 13) Создать иерархию классов – место, область, город, мегаполис
- 14) Создать иерархию классов – квитанция, накладная, документ, чек
- 15) Создать иерархию классов – соленая вода, море, залив, фьорд, лагуна
- 16) Создать иерархию классов – электроприбор, компьютеры, телевизоры, планшеты, утюги
- 17) Создать иерархию классов – зоопарк и его обитатели разных видов
- 18) Создать иерархию классов – описание иерархии карточных игр, можно придумать свою иерархию (например свинтус как развитие уно)

Приложение С. Обработка исключительных ситуаций

Задание 1 .

Для задания из первой лабораторной работы добавить обработку исключений типа `int`, `char *`, `float` (в зависимости от использованных типа данных) . Сгенерировать исключительную ситуацию.

Задание 2.

Добавить к программе перехват любой исключительной ситуации.

Задание 3.

Создать тип ошибка – `error`. Добавить к программе в задании 1 перехват исключительной ситуации типа, созданного в задании

Задание 4.

Создать тип ошибок памяти и тип ошибок с файлами, наследуемые от `error`. Добавить к программе в задании 1 перехват исключительной ситуации созданных типов.

Подписано в печать 14.01.19. Формат 60х90 (1/16)
Бумага офсетная № 1. Гарнитура «Times New Roman».
Печать офсетная. Усл. печ. л. 2,3 + обложка.
Тираж 100. Заказ 003.

Издательство: ЗАО ЛА «Научная книга»
410031, г. Саратов, ул. Волжская 28.

Сверстано и отпечатано в типографии «Автор»
410004, г. Саратов, ул. Советская 90/96,
тел. (8452) 24-85-92
e-mail: avtorprint@yandex.ru